

LZ AGENT

DOCUMENTO MESTRE COMPLETO - 2026

Windows • Android • Linux • IA multimodal • Memória persistente • Acessibilidade • 3D • Codex



LZ AGENT

SEU AGENTE PESSOAL INTELIGENTE
PARA TODOS, EM QUALQUER LUGAR!

Windows Android

Uma série de vídeos completa para ensinar como criar do zero um agente pessoal inteligente, acessível, integrado com todas as tecnologias e funcionalidades atuais e futuras, para ajudar em todos os tipos de empregos, serviços e para todas as pessoas, incluindo pessoas com deficiências.

CANAL: @lzworldsgames



1. VISÃO GERAL DO PROJETO

O LZ Agent é um agente pessoal inteligente, multimodal, acessível e personalizável, que utiliza o poder das melhores tecnologias atuais e futuras para ajudar em todos os tipos de tarefas, empregos e serviços. Ele é projetado para funcionar perfeitamente em Windows e Android, em qualquer dispositivo e hardware, com total acessibilidade para todas as pessoas.

PRINCÍPIOS

- Universal: Funciona em todos os dispositivos
- Acessível: Para todas as pessoas, sem exceção
- Inteligente: IA avançada e aprendizado contínuo
- Personalizável: Adaptável a qualquer necessidade
- Seguro: Privacidade e proteção de dados
- Integrado: Conectado com tudo ao seu redor

2. FUNCIONALIDADES PRINCIPAIS

- Assistente Inteligente Multimodal: Voz, Texto, Imagem, Vídeo e Gestos
- Automação de Tarefas: Execução automática de tarefas repetitivas
- Suporte a Todos os Empregos e Serviços: Soluções para todas as áreas e profissões
- Acessibilidade Total: Recursos para todos os tipos de deficiência
- Aprendizado Contínuo: Atualizações automáticas com novas IAs, tecnologias e linguagens
- Integração Total: Windows, Android, Web, IoT e mais
- Personalização Avançada: Interface, comandos, respostas e mais
- Segurança e Privacidade: Criptografia, controle de dados e permissões

3. TECNOLOGIAS E LINGUAGENS

INTELIGÊNCIA ARTIFICIAL

- OpenAI
- Claude
- Gemini
- Llama
- ...

LINGUAGENS

- Python
- Kotlin
- C#
- C++
- Rust
- Dart
- ...

DESENVOLVIMENTO

- .NET
- TensorFlow
- PyTorch
- Flutter
- Electron
- ...

BANCO DE DADOS

- SQLite
- Firestore
- PostgreSQL
- MongoDB
- ...

3D E MULTIMÍDIA

- Blender
- Unity
- Unreal
- Three.js
- ...

OUTROS

- Docker
- Git
- WebRTC
- MQTT
- ...

4. ACESSIBILIDADE TOTAL

O LZ Agent é projetado para ser utilizado por todas as pessoas, incluindo aquelas com deficiências visuais, auditivas, motoras, cognitivas e outras.

- Leitor de Tela Integrado
- Reconhecimento de Voz Avançado
- Comandos por Gestos e Movimentos
- Alto Contraste e Temas Personalizáveis
- Legendas Automáticas e Tradução
- Facilitação de Comunicação (AAC)
- Navegação Simplificada
- E muito mais...



5. INTEGRAÇÃO E COMPATIBILIDADE

Funciona em todos os hardwares e dispositivos:

- Windows: Todos os modelos e fabricantes
- Android: Todos os modelos e fabricantes
- Integração com IoT, Smart Devices, Robótica, Impressoras 3D e muito mais
- Sincronização entre dispositivos
- Funciona Offline e Online

6. INTERFACE DO AGENTE





7. PERSONALIZAÇÃO E 3D



8. SÉRIE DE VÍDEOS

A série será dividida em módulos completos, do zero ao avançado, cobrindo tudo que é necessário para criar, personalizar e manter o LZ Agent sempre atualizado.

MÓDULOS:

- 01 - Introdução ao Projeto
- 02 - Ambiente de Desenvolvimento
- 03 - Criação da Interface
- 04 - Integração com IA
- 05 - Comandos de Voz e Reconhecimento
- 06 - Automação de Tarefas
- 07 - Acessibilidade
- 08 - Integração com Hardware e IoT
- 09 - Sincronização entre Dispositivos
- 10 - Modelagem 3D do Agente
- 11 - Animações e Interações
- 12 - Publicação e Distribuição
- 13 - Atualizações e Manutenção
- 14 - Projetos Práticos e Exemplos



9. LOGO E ÍCONE



LZ AGENT - INTELIGÊNCIA, ACESSIBILIDADE E INOVAÇÃO PARA TODOS!

CANAL: youtube.com/@lzworldsgames

Plano unificado da série LZWORLDSGAMES e especificação oficial do agente pessoal

Consolida todas as decisões, mudanças, arquiteturas, imagens e requisitos definidos desde o início do projeto.

LZ AGENT

Plano mestre da série de vídeos para o canal LZWORLDSGAMES

Criação do zero de um agente pessoal multimodal, acessível, extensível, com memória persistente em banco de dados e integrado ao Windows, Android e Linux

Documento mestre consolidado • revisão 29/08/2026 • Windows + Android + Linux • banco de dados, memória persistente, aprendizado controlado e especificação oficial do modelo

1. Visão do projeto

A série acompanhará a construção real de um agente pessoal chamado provisoriamente de LZ Agent. O objetivo não é criar apenas um chatbot: o sistema será uma plataforma de assistência capaz de conversar, entender voz, texto, imagens e documentos, executar tarefas autorizadas, organizar rotinas, auxiliar atividades profissionais, apoiar criação de software e conteúdo, trabalhar com modelos 2D/3D e adaptar sua interface às necessidades de acessibilidade de cada pessoa.

A proposta deve ser desenvolvida de forma modular e verificável. Nenhum episódio deve prometer compatibilidade literal com todo hardware existente. Em vez disso, o projeto usará níveis de compatibilidade, detecção de recursos e modos de desempenho para alcançar a maior variedade prática de PCs Windows, distribuições Linux e aparelhos Android.

Princípios obrigatórios

- Acessibilidade por padrão, e não como complemento.
- Privacidade, consentimento e permissões mínimas para qualquer automação.
- Arquitetura modular: cada recurso pode ser instalado, atualizado, desativado ou substituído.
- Funcionamento online e, quando tecnicamente possível, modo local/offline.
- Escalonamento de desempenho: Lite, Standard e Pro.
- Código versionado, documentado, testado e reproduzível em cada episódio.
- Uso do Codex como parceiro de engenharia, com revisão humana, testes e validação antes de integrar código.
- Atualização contínua de dependências, linguagens, modelos e APIs sem reescrever todo o produto.
- Memória persistente: toda ação executada pelo agente gera um registro auditável no banco de dados, respeitando privacidade e permissões.

- **Aprendizado controlado:** o agente transforma resultados, correções e preferências autorizadas em conhecimento reutilizável para projetos futuros.
- **Sugestões evolutivas:** o agente compara projetos anteriores e atuais para recomendar melhorias, reutilização de soluções, riscos e próximos passos.
- **Mudanças seguras:** alterações automáticas em projetos só podem ocorrer dentro de escopos aprovados, sempre com versão anterior, diff, teste e rollback.

2. Público e tipos de trabalho atendidos

Área	Exemplos de ajuda do agente
Administrativo	agenda, documentos, e-mails, relatórios, planilhas, reuniões e pesquisa
Comércio e serviços	cadastros, atendimento, estoque, orçamento, lembretes e organização
Programação/TI	planejamento, geração assistida de código, testes, documentação, Git e diagnóstico
Design/3D	briefings, assets, Blender, materiais, animação, exportação e organização de projetos
Educação	resumos, estudo guiado, acessibilidade de conteúdo, exercícios e organização
Criação de conteúdo	roteiros, thumbnails, legendas, organização de mídia e publicação assistida
Manutenção/técnico	checklists, manuais, inventário, diagnóstico orientado e relatórios
Profissionais autônomos	clientes, propostas, agenda, finanças simples e acompanhamento de tarefas

Para profissões reguladas ou de alto risco, como saúde, direito, segurança e finanças, o agente deve atuar como assistente de informação e fluxo de trabalho, mantendo revisão humana e limites claros.

3. Acessibilidade universal como requisito de engenharia

O produto será desenhado para diferentes combinações de deficiência visual, auditiva, motora, de fala, cognitiva e de aprendizagem. A meta de referência para interfaces web será WCAG 2.2 nível AA, complementada pelas APIs nativas de acessibilidade do Windows, Android e Linux.

Necessidade	Recursos planejados
Visual	leitor de tela, foco lógico, alto contraste, escala de texto, zoom, descrição de imagens, áudio
Auditiva	legendas, transcrição em tempo real, alertas visuais e vibração/háptica
Motora	controle por voz, teclado, switch, atalhos, alvos grandes, tempo ajustável, automação
Fala	entrada por texto, frases rápidas, comunicação aumentativa e alternativa (AAC)
Cognitiva	modo simples, linguagem clara, etapas curtas, confirmação de ações e redução de distrações
Múltipla	perfis combináveis, preferências sincronizadas e fallback entre modalidades

Checklist de acessibilidade por episódio

- Tudo que pode ser feito por toque/mouse também deve possuir alternativa por teclado, voz ou ação acessível quando aplicável.
- Elementos interativos têm nome, função, estado e ordem de foco compreensíveis.
- Não depender apenas de cor, som, gesto ou animação para transmitir informação.
- Permitir reduzir/desativar animações e efeitos.
- Testar com Narrator/teclado no Windows e TalkBack, Voice Access e Switch Access no Android.
- Legendar todos os vídeos da série e explicar visualmente e verbalmente o que está acontecendo.

4. Arquitetura técnica proposta

A arquitetura separa interface, núcleo do agente, conectores, ferramentas, memória, modelos e automação. Isso permite trocar uma tecnologia sem quebrar o restante do sistema.

Camada	Responsabilidade	Tecnologias candidatas
UI Windows	desktop, acessibilidade, notificações, voz, configurações	C#/.NET + WinUI 3 / Windows App SDK
UI Android	mobile, acessibilidade, voz, sensores, notificações	Kotlin + Jetpack Compose
Core do agente	orquestração, planejamento, ferramentas, políticas	Python e/ou Rust
Serviços locais	IPC, tarefas, filas, cache, modelos	Python 3.14.7 + FastAPI 0.141.1, gRPC/HTTP local e Rust 1.98 quando necessário
IA	LLM, visão, fala, embeddings, roteamento	APIs de IA + modelos locais compatíveis
Dados	configuração, memória, ações, projetos, aprendizado, auditoria e histórico autorizado	SQLite 3.53.4 local + PostgreSQL 18.6 opcional para sincronização/servidor
Automação	ações no SO e apps com consentimento	APIs nativas, conectores e plugins
3D/mídia	criação e manipulação de assets	Blender Python API, FFmpeg, formatos glTF
UI Linux	desktop, acessibilidade, notificações, voz, arquivos e integração cross-distro	Flutter/GTK ou toolkit equivalente + Flatpak/Portals
Motor de memória/aprendizado	registrar ações, extrair aprendizados, comparar projetos, gerar sugestões e manter versões	SQL + FTS/embeddings opcionais + event log + snapshots Git

O núcleo não deve depender diretamente de uma única empresa de IA. Uma interface de provedores permitirá trocar modelos em nuvem ou locais, selecionar custo/latência/privacidade e manter o projeto sustentável.

5. Tecnologias e linguagens

Categoria	Base recomendada	Uso
Windows	C# 14 + .NET 10 + XAML + WinUI 3 /	aplicativo nativo e integração com

Categoria	Base recomendada	Uso
	Windows App SDK 2.4.0	o sistema
Android	Kotlin 2.4.10 + Jetpack Compose + AGP 9.3.2	aplicativo nativo e acessível
IA/automação	Python 3.14.7 + FastAPI 0.141.1	orquestração, ML, visão, voz e scripts
Alto desempenho	Rust 1.98.0 / C++ quando necessário	módulos críticos, bindings e inferência
Web/painéis	TypeScript, HTML, CSS	painel opcional, documentação e serviços
3D	Python 3.14.7 + Blender 5.2.1 LTS API	modelagem procedural, rig, materiais e exportação
Dados	SQL: SQLite 3.53.4 + PostgreSQL 18.6	memória local, histórico, projetos, auditoria, sincronização opcional
Interoperabilidade	JSON, Protobuf, REST/gRPC	contratos entre módulos
Linux	Flutter 3.47.0 + Dart 3.13.0 + Flatpak 1.18.x/linha estável da distro	desktop cross-distro, Flatpak, Portals e integrações do sistema

A série deve ensinar por que cada linguagem existe no projeto. 'Usar todas as linguagens' não é uma meta técnica saudável; a meta é suportar novas linguagens por plugins/SDKs e escolher a ferramenta adequada para cada módulo.

6. Compatibilidade de hardware

Perfil	Objetivo	Estratégia
Lite	hardware antigo/entrada	UI leve, IA em nuvem opcional, animações reduzidas, modelos locais pequenos
Standard	PC/celular intermediário	recursos multimodais equilibrados e cache local
Pro	hardware forte/NPU/GPU	modelos locais maiores, visão contínua autorizada, 3D e aceleração

No Linux, a compatibilidade será tratada por matriz de distribuição + versão + desktop + Wayland/X11 + arquitetura. Flatpak será o pacote principal porque seus runtimes são independentes da distribuição e fornecem uma base estável entre distros; integrações privilegiadas continuarão condicionadas às permissões e Portals disponíveis.

- Windows: detectar x64/x86/ARM64 quando suportado pelo componente e validar versões mínimas do sistema.
- Android: adaptar layouts a telas, densidades, tablets, dobráveis e diferentes fabricantes; evitar dependências de uma marca.
- GPU/NPU são acelerações opcionais; o agente precisa ter fallback para CPU ou nuvem quando permitido.
- Cada release terá uma matriz de dispositivos testados, emuladores, requisitos mínimos e problemas conhecidos.

7. Segurança, privacidade e controle do usuário

- Permissões por capacidade: microfone, câmera, arquivos, contatos, calendário, localização e automação separadas.
- Confirmação explícita para ações destrutivas, compras, envio de mensagens, exclusões e alterações sensíveis.
- Cofre local de segredos usando mecanismos seguros do sistema operacional; nunca gravar chaves em código.
- Logs compreensíveis: o usuário pode ver o que o agente fez, por quê e com qual ferramenta.
- Modo privado/local quando possível, política de retenção e botão para apagar memória/histórico.
- Sandbox para ferramentas e lista de ações permitidas por plugin.
- Atualizações assinadas, verificação de integridade, dependências auditadas e testes de segurança.

8. Sistema de plugins e profissões

A universalidade será alcançada por pacotes de capacidade. O núcleo continua pequeno e os módulos adicionam conhecimento operacional, integrações e interfaces específicas.

Plugin	Exemplos
Office/Produtividade	documentos, planilhas, PDF, agenda, e-mail, reuniões
Dev	terminal, Git, IDE, testes, documentação, ambientes virtuais, build e diagnóstico
3D/Design	Blender, assets, render, conversão e catálogos
Mídia	áudio, vídeo, transcrição, legendas e organização
Atendimento	CRM, respostas, formulários e acompanhamento
Acessibilidade	AAC, macros por voz, leitura, simplificação e perfis
Casa/IoT	dispositivos compatíveis, cenas e rotinas com consentimento
Memória/Projetos	histórico de ações, decisões, erros, soluções, sugestões, versões e reutilização entre projetos

Cada plugin terá manifesto, versão, permissões, comandos, esquema de entrada/saída, testes e documentação. Plugins de terceiros nunca recebem acesso irrestrito ao sistema.

9. Modelagem 3D, avatar e animações

O agente terá identidade visual própria e poderá usar avatar 2D/3D. O pipeline principal de 3D será construído com Blender e scripts Python para automatizar tarefas repetitivas.

1. Concept art e guia de proporções do avatar.
2. Modelagem low-poly e versão de maior detalhe.
3. UV, materiais e texturas.
4. Rig do corpo e, se necessário, rig facial.
5. Animações: idle, ouvir, falar, pensar, confirmar, erro, comemorar e modo acessibilidade.
6. LOD e otimização para diferentes hardwares.
7. Exportação em glTF/GLB e assets específicos para Windows/Android/Linux.

8. Fallback 2D/estático para aparelhos de baixo desempenho ou usuários que desativem animações.

10. Identidade visual: logo e ícone

A direção visual sugerida usa um núcleo circular/visor luminoso como símbolo de inteligência, com as letras LZ integradas ao centro. A marca deve funcionar em quatro versões: completa, horizontal, ícone quadrado e monocromática.

- Ícone: forma simples, reconhecível em 16–32 px e sem texto pequeno.
- Animação: pulso de luz ao ouvir, onda ao falar, rotação suave ao processar e estado reduzido sem movimento.
- Acessibilidade: animação opcional; contraste suficiente; nenhum estado comunicado apenas por cor.
- Pacote final: SVG/PNG, ícones Windows, adaptive icon Android, splash e assets do canal.

11. Interface principal

Área	Função
Home	campo de comando, voz, sugestões e estado do agente
Tarefas	fila, progresso, aprovações e histórico
Ferramentas	plugins instalados e permissões
Memória	ações registradas, conhecimento aprendido, projetos, fontes, sugestões, edição, exportação e exclusão
Acessibilidade	perfil, leitor, contraste, tamanho, voz, switch, animações
Dispositivos	Windows/Android/Linux conectados e sincronização
Desenvolvedor	logs, testes, modelos, API local e diagnóstico

A interface deve oferecer modo simples e modo avançado. O modo simples prioriza uma ação por vez e linguagem clara; o avançado mostra ferramentas, logs, modelos e automações.

12. Integração Windows

- Aplicativo nativo com Windows App SDK/WinUI 3.
- Inicialização opcional com o sistema, bandeja/área de notificação e atalhos globais quando permitido.
- Notificações, compartilhamento, arquivos e protocolos/deep links.
- Integração com Narrator, teclado, escala de texto, alto contraste e preferências de movimento.
- Serviço local separado da UI para tarefas longas, com autenticação IPC.
- Instalação e atualização com estratégia compatível com arquiteturas suportadas.

13. Integração Linux

- Aplicativo desktop Linux com camada de interface multiplataforma, mantendo o núcleo do agente independente da distribuição.
- Distribuição principal via Flatpak para reduzir diferenças entre distros, com runtimes, sandbox e Portals; pacotes nativos poderão ser adicionados quando houver necessidade real.
- Meta de compatibilidade: Ubuntu, Debian, Linux Mint, Fedora, openSUSE, Arch/Manjaro, Pop!_OS, elementary OS, Zorin OS, EndeavourOS, Alpine e outras distribuições modernas, sem afirmar suporte irrestrito antes dos testes.
- Ambientes gráficos: GNOME, KDE Plasma, Cinnamon, Xfce, MATE, LXQt e outros, respeitando padrões Freedesktop e integração de desktop.
- Sessões Wayland e X11 serão tratadas por uma camada de capacidades; recursos dependentes do compositor terão fallback ou serão desativados com explicação clara.
- Acessibilidade Linux: integração com tecnologias e padrões disponíveis no desktop, navegação completa por teclado, contraste, escala, redução de movimento, leitores de tela e alternativas multimodais.
- Integrações com arquivos, notificações, abertura de URLs, câmera, microfone e compartilhamento devem preferir XDG Desktop Portals quando o app estiver em sandbox.
- Arquiteturas de CPU serão declaradas por release e testadas, priorizando x86_64 e ARM64/aarch64 quando todas as dependências forem compatíveis.
- O projeto terá scripts de diagnóstico para detectar distro, kernel, arquitetura, sessão gráfica, GPU, aceleração disponível e recursos de acessibilidade antes de habilitar módulos pesados.

14. Integração Android

- Kotlin + Jetpack Compose e arquitetura em camadas.
- UI adaptativa para celulares, tablets e dobráveis.
- TalkBack, Voice Access, Switch Access, semântica Compose e alvos de toque adequados.

- Foreground services apenas quando necessários e claramente explicados.
- WorkManager para tarefas adiáveis e notificações para acompanhamento.
- Permissões solicitadas no contexto certo; evitar AccessibilityService como atalho para automação genérica.

15. IA multimodal e memória

Componente	Responsabilidade
Roteador	escolher modelo/ferramenta conforme tarefa, privacidade, custo e hardware
LLM	compreensão, planejamento, explicação e geração
STT	fala para texto
TTS	texto para fala
Visão	entender imagens/câmera quando autorizado
Embeddings	busca semântica em memória/documentos autorizados
Memória	armazenar histórico autorizado, decisões, preferências, resultados, projetos e aprendizados com transparência, retenção e exclusão
Tool runner	executar ações permitidas e devolver resultados estruturados
Action Ledger	registrar cada ação do agente, entrada, ferramenta, resultado, horário, projeto, permissão e status
Project Memory	consolidar arquitetura, decisões, erros, soluções, padrões, assets e próximos passos de cada projeto
Learning Engine	transformar feedback e resultados em conhecimento reutilizável sem alterar modelos-base por conta própria
Suggestion Engine	comparar histórico e contexto para recomendar mudanças, melhorias e soluções para projetos futuros
Version/rollback	criar checkpoints e diffs antes de mudanças para permitir revisão, aprovação e restauração

O agente nunca deve executar mudanças sensíveis sem política de autorização. Planejamento, memória e execução são separados: o modelo propõe; a camada de políticas valida permissões; o sistema registra a intenção no banco; pede confirmação quando necessário; executa; salva resultado, diff e métricas; e então atualiza somente os aprendizados autorizados.

15.1 Memória persistente, aprendizado contínuo e banco de dados

O LZ Agent terá uma memória operacional própria. O estado durável do agente será mantido diretamente em banco de dados, com migrações, backups, auditoria e controle de retenção. SQLite será o padrão local em Windows, Android e Linux; PostgreSQL será opcional para sincronização entre dispositivos, equipes ou serviços remotos.

“Aprender” significa construir uma base de conhecimento controlada pelo usuário a partir das ações do próprio agente: o que funcionou, o que falhou, quais correções foram aprovadas, preferências, decisões de arquitetura, padrões de código, assets, etapas de trabalho, desempenho e feedback. Isso não significa retreinar silenciosamente um modelo-base nem coletar indiscriminadamente todos os dados da pessoa.

Cada projeto possuirá memória separada e também poderá contribuir, mediante permissão, para uma memória global do usuário. Assim, ao iniciar um projeto novo, o agente poderá sugerir componentes já usados, alertar sobre erros recorrentes, propor melhorias de arquitetura, estimar riscos e reaproveitar soluções aprovadas anteriormente.

O agente poderá aplicar mudanças automaticamente somente quando o usuário ativar esse nível de autonomia para uma pasta, projeto ou tipo de ação. Antes de qualquer alteração relevante, será criado um checkpoint; depois, serão gerados diff, testes e relatório. Se algo falhar, o sistema poderá reverter para a versão anterior.

Política de retenção e privacidade

- Por padrão, registrar tudo que o agente FAZ; não registrar indiscriminadamente tudo que o usuário vê, digita, fala ou possui.
- Dados sensíveis e conteúdo de arquivos só entram na memória quando necessários e autorizados.
- Permitir memória temporária, por projeto, permanente e modo privado sem retenção.
- Permitir pesquisar, editar, exportar e apagar memórias e histórico.
- Criptografar dados locais sensíveis e proteger credenciais com o cofre do sistema operacional.
- Sincronização entre dispositivos deve ser opcional, autenticada e criptografada.
- Sugestões são separadas de ações: recomendar não significa executar automaticamente.

Tabela/coleção	O que armazena	Objetivo
agent_actions	comando, ferramenta, parâmetros permitidos, resultado, erro, duração e status	auditoria completa das ações
projects	metadados, stack, caminhos, objetivos, estado e versões	memória por projeto
project_decisions	decisões técnicas, motivo, alternativas e resultado	evitar repetir decisões ruins
lessons_learned	problema, solução, contexto, evidência e confiança	aprendizado reutilizável
suggestions	melhoria sugerida, prioridade, impacto, justificativa e decisão do usuário	evolução contínua
artifacts	referências a documentos, código, imagens, modelos 3D e exports	rastrear o que foi criado
user_preferences	preferências autorizadas de interface, acessibilidade e fluxo	personalização persistente
memory_sources	origem, data, consentimento, retenção e escopo	governança de memória
checkpoints	commit/hash, arquivos afetados, diff e resultado dos testes	rollback e segurança
tech_versions	dependência, versão aprovada, data de verificação e origem	manter a stack atualizada

15.2 Especificação oficial do modelo interno do LZ Agent

Esta seção é a especificação canônica do agente que será implementado dentro do produto. Qualquer protótipo, episódio, interface, avatar, plugin ou atualização deverá respeitar estes contratos. As artes conceituais antigas deixam de ser a fonte de verdade quando conflitarem com esta especificação; a

fonte de verdade passa a ser este documento, os arquivos de configuração versionados e os testes automatizados.

Aspecto	Configuração oficial	Regra de fidelidade
Nome do produto	LZ Agent	usar o mesmo nome em Windows, Android e Linux
Função	agente pessoal multimodal, acessível, extensível e orientado a tarefas	não reduzir o produto a um chatbot
Plataformas	Windows + Android + Linux	mesma identidade, memória e contratos; integrações variam por SO
Entrada	texto, voz, imagem, documentos, atalhos, gestos acessíveis e eventos autorizados	sempre oferecer alternativa acessível quando aplicável
Saída	texto, voz, notificações, ações, arquivos, código, mídia, modelos 2D/3D e relatórios	toda ação deve ser rastreável
Memória	SQLite local por padrão + PostgreSQL opcional para sincronização	memória transparente, editável, exportável e apagável
Aprendizado	lessons learned + feedback + resultados + confiança	não alterar modelo-base silenciosamente; aprender via memória controlada
Autonomia	graduada por perfil e permissão	mudanças relevantes exigem checkpoint, diff, teste e rollback
Acessibilidade	perfil combinável por usuário	não depender de uma única modalidade sensorial/motora
Identidade visual	núcleo circular/visor luminoso com marca LZ; avatar 2D/3D opcional	mesmos estados semânticos em todas as plataformas

15.3 Configuração dos modelos de IA e roteamento

O LZ Agent usará uma camada Provider Router. O código nunca deve depender de um único modelo. O roteador escolhe o modelo com base em tarefa, privacidade, custo, latência, hardware, disponibilidade e necessidade de ferramentas. Em 29/08/2026, a referência de produção para OpenAI é gpt-5.6-sol para raciocínio complexo e gpt-5.3-codex para programação agentic; os IDs ficam em configuração e podem ser substituídos sem alterar o núcleo.

Parâmetro	Padrão	Comportamento
provider.default	openai	pode ser trocado por provedor local/nuvem compatível
model.general	gpt-5.6-sol	conversação, planejamento, visão e tarefas complexas
model.coding	gpt-5.3-codex	código, refatoração, testes, análise de repositório e implementação
reasoning.default	medium	eleva para high/xhigh em tarefas complexas; reduzir em tarefas simples
temperature/creativity	perfil por tarefa	baixa em automação/código; maior em criação, sem sacrificar precisão
tool_policy	allowlist + permissões	modelo só vê ferramentas autorizadas para aquele contexto
context_policy	mínimo necessário	buscar memória/documentos somente quando relevantes e permitidos

Parâmetro	Padrão	Comportamento
fallback	local -> nuvem ou nuvem -> local conforme política	degradação graciosa em falhas, offline ou hardware limitado
model_manifest	versionado no banco e no repositório	registrar modelo, data, provedor e configuração usada em cada ação

15.4 Personalidade, comunicação e voz

A personalidade oficial do LZ Agent será prestativa, clara, respeitosa, objetiva e adaptável. Ele deve explicar decisões quando solicitado, indicar incerteza, não fingir que executou uma ação que falhou e nunca pressionar o usuário a conceder permissões. O nível de detalhe, ritmo de fala, vocabulário e forma de apresentação podem ser ajustados por perfil.

Configuração	Valores previstos	Padrão
Tom	objetivo, amigável, técnico, simples, didático	amigável + claro
Detalhamento	curto, equilibrado, completo	equilibrado
Idioma	detecção automática + idioma fixo por perfil	Português (pt-BR) quando selecionado
Velocidade TTS	0,5x a 2,0x	1,0x
Voz	sistema/local/nuvem conforme disponibilidade	voz do sistema com opção de troca
Leitura automática	desligada, respostas importantes, todas	desligada por padrão
Confirmações	visual, sonora, háptica, fala	visual + modalidade escolhida pelo usuário
Linguagem simples	automática ou manual	disponível em qualquer tela/resposta

15.5 Avatar 2D/3D e estados de animação

O avatar é uma camada de apresentação, nunca requisito para o funcionamento do agente. A forma oficial usa um núcleo/visor escuro de aspecto tecnológico com luz ciano/azul e identificação LZ. Pode existir como ícone, esfera/robô compacto 3D ou personagem simplificado, mas os estados abaixo são obrigatórios e semanticamente consistentes. Pessoas que ativarem redução de movimento recebem transições estáticas ou discretas.

Estado	Visual/animação	Significado
Idle	pulso suave ou ícone estático	pronto e aguardando
Listening	anel/onda de entrada	microfone ativo por autorização
Thinking	movimento circular/progresso	processando/planejando
Acting	indicador de ferramenta/tarefa	executando ação autorizada
Speaking	onda sincronizada com TTS	resposta por voz
Needs approval	símbolo claro de confirmação	aguardando consentimento
Success	confirmação curta não dependente só de cor	ação concluída
Warning	alerta visual + texto/voz/háptica configurável	risco ou atenção
Error	estado de falha + mensagem acessível	ação não concluída
Offline	indicador local	sem rede; recursos limitados ou locais
Private	ícone de privacidade	sessão sem retenção quando configurado

15.6 Perfis de autonomia e mudanças em projetos

O agente pode sugerir e, quando explicitamente autorizado, realizar mudanças em projetos. A autonomia não é binária: cada pasta, repositório, profissão, plugin e tipo de ação possui sua própria política.

Nível	Permissão	Exemplo
0 - Consultivo	somente responde e sugere	explicar melhoria sem alterar arquivos
1 - Preparar	gera plano, patch/diff ou artefato sem aplicar	criar proposta de alteração
2 - Aplicar com confirmação	aplica após aprovação explícita	editar código após mostrar diff
3 - Autonomia limitada	executa ações previamente aprovadas dentro de escopo	formatar, testar, atualizar documentação
4 - Fluxo automatizado supervisionado	executa cadeia permitida e pede aprovação em pontos sensíveis	corrigir, testar, gerar build e apresentar release

- Antes de alterações relevantes: criar checkpoint Git ou snapshot equivalente e registrar no banco.
- Depois da alteração: salvar diff, comandos executados, testes, resultado, tempo e modelo de IA usado.
- Falha em teste obrigatório bloqueia a promoção da mudança e oferece rollback.
- Sugestões futuras devem citar qual projeto, decisão ou lição anterior motivou a recomendação.
- Mudanças destrutivas, publicação, envio de mensagens, pagamentos e acesso sensível nunca entram em autonomia irrestrita.

15.7 Memória, aprendizado e banco de dados - configuração oficial

O LZ Agent registra o que faz para aprender com o próprio histórico sem treinar ou modificar secretamente o modelo-base. O aprendizado ocorre por memória estruturada, recuperação semântica opcional, regras, feedback e avaliação de resultados. O usuário controla retenção, escopo e exclusão.

Tipo de memória	Escopo	Retenção padrão	Uso
Session memory	sessão atual	até encerrar ou prazo configurado	continuidade imediata
Project memory	um projeto/repositório	persistente até exclusão	decisões, erros, soluções, assets e próximos passos
User preferences	usuário	persistente	acessibilidade, interface, idioma e hábitos autorizados
Lessons learned	projeto ou global autorizado	persistente com confiança/evidência	reutilizar soluções e evitar erros recorrentes
Action ledger	todas as ações autorizadas	política configurável	auditoria e explicabilidade
Private session	sessão privada	não persistir conteúdo, salvo logs mínimos de segurança quando exigidos	tarefas sensíveis

Banco local: SQLite com foreign keys, WAL quando apropriado, migrações versionadas, índices de busca e criptografia/segurança baseada nos mecanismos suportados pela plataforma. Sincronização opcional: PostgreSQL, com conflitos resolvidos por versão, dispositivo, timestamp e regra de precedência. O banco nunca substitui backup: haverá exportação e cópias verificadas.

15.8 Configurações de acessibilidade do próprio agente

Grupo	Configurações
Visual	escala, contraste, tema, fonte, espaçamento, foco, leitor de tela, descrição de imagens, reduzir transparência
Auditiva	legendas, transcrição, alertas visuais, vibração/háptica, repetição e histórico textual
Motora	voz, teclado, switch, dwell, atalhos, tamanho de alvo, duração de pressão, tempo de confirmação
Cognitiva	modo simples, uma ação por tela, linguagem clara, confirmações, lembretes, ocultar elementos avançados
Fala/AAC	frases rápidas, pranchas, entrada alternativa, TTS, histórico reutilizável
Movimento	reduzir/desativar animações, parallax e efeitos piscantes
Leitura	velocidade, destaque de frase/palavra, pausa, pronúncia e idioma

15.9 Arquivo de configuração canônico

O repositório terá config/agent.defaults.json como padrão versionado e um arquivo local de preferências por usuário. Segredos ficam fora desses arquivos. Um exemplo mínimo de contrato é:

```
{
  "schema_version": 1,
  "agent": {
    "name": "LZ Agent",
    "language": "pt-BR",
    "autonomy_level": 1
  },
  "providers": {
    "general": {
      "provider": "openai",
      "model": "gpt-5.6-sol",
      "reasoning": "medium"
    },
    "coding": {
      "provider": "openai",
      "model": "gpt-5.3-codex",
      "reasoning": "high"
    }
  },
  "memory": {
    "backend": "sqlite",
    "project_memory": true,
    "lessons_learned": true,
    "action_ledger": true,
    "private_mode_retention": false
  },
  "database": {
    "local_file": "data/lz_agent.db",
    "migrations": true,
    "backup": true,
    "remote_sync": "disabled"
  },
  "ui": {
    "mode": "simple",
    "theme": "system",
    "animations": "reduced-when-requested"
  }
}
```



```

},
"accessibility": {
  "screen_reader": "system",
  "captions": true,
  "keyboard_full_access": true,
  "voice_control": true,
  "switch_access": true
},
"safety": {
  "confirm_destructive_actions": true,
  "checkpoint_before_project_change": true,
  "rollback_on_failed_tests": true
},
"avatar": {
  "enabled": true,
  "render_mode": "auto",
  "fallback": "2d-static"
}
}

```

15.10 Fidelidade entre plataformas

Windows, Android e Linux não precisam usar o mesmo toolkit visual, mas precisam apresentar os mesmos conceitos: Home, Tarefas, Ferramentas, Memória, Acessibilidade, Dispositivos e Desenvolvedor; os mesmos estados do agente; as mesmas regras de permissão; e o mesmo formato de memória. Cada adaptação nativa será considerada correta somente se passar por testes de contrato e acessibilidade.

15.11 Critérios para considerar o modelo 100% fiel

Critério	Teste de aceitação
Identidade	nome, ícone, estados e comportamento correspondem à especificação oficial
IA	roteamento usa manifest/versionamento e registra o modelo de cada execução
Memória	ações, projetos, lições e sugestões persistem conforme política escolhida
Aprendizado	sugestões futuras mostram origem/evidência e podem ser rejeitadas/apagadas
Mudanças	checkpoint + diff + testes + rollback disponíveis antes/depois de alterações
Acessibilidade	fluxo principal utilizável pelas modalidades suportadas sem depender de cor/som/gesto isolado
Cross-platform	contratos de dados e estados idênticos em Windows, Android e Linux
Hardware	perfil Lite/Standard/Pro seleciona recursos sem quebrar função essencial
Privacidade	usuário visualiza, exporta e apaga dados; modo privado funciona como definido

16. Como o Codex entra no projeto

O Codex será usado como agente de programação para acelerar tarefas de engenharia, mas o repositório continuará sendo a fonte de verdade. Cada episódio deve mostrar o pedido feito ao Codex, os arquivos alterados, os testes, a revisão e a correção de eventuais falhas.

1. Definir issue e critérios de aceitação.
2. Pedir ao Codex uma implementação pequena e isolada.
3. Revisar diff e dependências.
4. Executar lint, testes unitários, integração e acessibilidade.
5. Testar manualmente em Windows/Android/Linux.
6. Documentar decisões e limitações.
7. Fazer commit somente após a validação.

17. Estrutura sugerida do repositório

```
lz-agent/  
├─ apps/  
│   ├── windows/  
│   ├── linux/  
│   └─ android/  
├─ core/  
│   ├── agent/  
│   ├── memory/  
│   ├── learning/  
│   ├── suggestions/  
│   ├── tools/  
│   ├── providers/  
│   └─ policies/  
├─ data/  
│   ├── migrations/  
│   ├── schemas/  
│   ├── repositories/  
│   └─ backups/  
├─ services/  
│   ├── voice/  
│   ├── vision/  
│   └─ local-api/  
├─ plugins/  
│   ├── productivity/  
│   ├── developer/  
│   ├── media/  
│   └─ blender/  
├─ assets/  
│   ├── brand/  
│   └─ avatar/
```

- | └─ audio/
- | └─ accessibility/
- └─ shared/
- | └─ schemas/
- | └─ localization/
- └─ tests/
- └─ docs/
- └─ scripts/
- └─ versions/
- | └─ technology-baseline.json
- └─ .github/

18. Plano da série de vídeos

Ep.	Título	Entrega principal
01	Manifesto e visão do LZ Agent	problema, público, acessibilidade, arquitetura e demonstração-alvo
02	Preparando o ambiente	Git, IDEs, SDKs, Python, .NET, Android Studio, Blender e organização
03	Monorepo e contratos	pastas, schemas, configuração, CI e testes
04	Núcleo do agente do zero	mensagens, sessões, tool calling, erros e logs
05	Provedores de IA	abstração de modelos, nuvem/local, roteamento e fallback
06	Banco de dados e memória persistente	SQLite, schema, migrações, histórico de ações, projetos, consentimento e exclusão
06B	Aprendizado contínuo controlado	lessons learned, feedback, confiança, memória global/projeto e prevenção de repetição de erros
06C	Motor de sugestões para projetos futuros	comparação de projetos, recomendações, priorização, justificativas e aprovação antes de mudanças
07	Voz	STT, TTS, wake/press-to-talk, ruído, legendas e privacidade
08	Visão e documentos	imagens, câmera autorizada, PDFs e descrição acessível
09	Windows: primeira interface	WinUI 3, navegação, tema e acessibilidade
10	Android: primeira interface	Compose, arquitetura, layouts adaptativos e acessibilidade
10B	Linux: primeira interface	desktop Linux, Flatpak, Portals, GNOME/KDE e acessibilidade
10C	Linux: compatibilidade entre distros	Debian/Ubuntu/Mint, Fedora, openSUSE, Arch/Manjaro e matriz de testes
10D	Linux: Wayland, X11 e integrações	sessões gráficas, notificações, arquivos, voz, permissões e

Ep.	Título	Entrega principal
		fallbacks
11	Sistema de ferramentas	manifestos, permissões, execução e sandbox
12	Automação segura + Action Ledger	aprovações, tarefas, registro de ações, checkpoints, rollback e auditoria
13	Plugin de produtividade	arquivos, documentos, agenda e rotinas
14	Plugin para desenvolvedores	terminal, Git, testes e assistência com Codex
15	Acessibilidade visual	screen readers, foco, contraste, zoom e descrição
16	Acessibilidade motora e voz	Voice Access, Switch, teclado, atalhos e tempos
17	Acessibilidade auditiva/cognitiva	legendas, alertas, modo simples e linguagem clara
18	Avatar e logo	identidade visual, estados, animação reduzida e assets
19	Blender e modelagem 3D	pipeline, scripts, rig, animação, LOD e glTF
20	Integração do avatar	renderização, sincronização labial/estado e fallback 2D
21	Perfis Lite/Standard/Pro em três sistemas	benchmark, detecção de hardware e seleção de recursos no Windows, Linux e Android
22	Offline e modelos locais	download, armazenamento, inferência e limitações
23	Sincronização Windows/Linux/Android	conta opcional, criptografia, pareamento e conflitos
24	Plugins por profissão + aprendizado	SDK, memória por domínio, sugestões, permissões e expansão futura
25	Segurança	segredos, threat model, dependências, assinatura e testes
26	Testes completos	unitário, integração, UI, acessibilidade, desempenho e dispositivos
27	Instaladores, Flatpak e atualização	Windows, Android, Linux/Flatpak, canais de release e rollback
28	Beta público	telemetria opcional, feedback, bugs e matriz de compatibilidade
29	Otimização final	latência, memória, bateria, rede e 3D
30	Versão 1.0 e futuro	release, documentação, roadmap e manutenção contínua

19. Estrutura de cada vídeo

1. Abertura de 15–30 s mostrando o que estará funcionando ao final.

- 2. Objetivo e pré-requisitos.
- 3. Diagrama rápido da parte que será construída.
- 4. Implementação em blocos pequenos.
- 5. Uso do Codex mostrando prompt, diff e revisão.
- 6. Teste funcional.
- 7. Teste de acessibilidade.
- 8. Teste em hardware/perfil diferente.
- 9. Erros encontrados e correções.
- 10. Commit/tag do episódio e resumo.
- 11. Desafio para o público e teaser do próximo episódio.

20. Padrão de qualidade para código

- Nenhum segredo no repositório; usar arquivos de ambiente e cofres adequados.
- Tipagem, lint e formatação automáticos.
- Testes para lógica crítica e contratos entre módulos.
- Mensagens de erro úteis e localizáveis.
- Logs e registros de ações estruturados no banco, com minimização de dados pessoais e políticas de retenção.
- Documentação de instalação reproduzível.
- Dependências fixadas/gerenciadas e atualização automatizada com testes.
- Benchmarks de CPU, RAM, bateria e latência nos perfis Lite/Standard/Pro.

21. Atualização contínua da série

Como IA, SDKs e linguagens mudam rapidamente, o projeto terá TECHNOLOGY_RADAR.md e versions/technology-baseline.json. Antes de cada gravação e release, um verificador consulta as fontes oficiais, registra a versão estável aprovada, executa os testes e somente então abre uma atualização. O agente poderá sugerir upgrades automaticamente, mas mudanças de versão só serão aplicadas após validação de compatibilidade, migrações de banco, testes de acessibilidade, desempenho e rollback.

22. Métricas de sucesso

Dimensão	Métrica
Acessibilidade	fluxos principais concluídos com teclado/screen reader/voz/switch conforme plataforma
Compatibilidade	matriz crescente de dispositivos e zero crash crítico

Dimensão	Métrica
	nos perfis suportados
Desempenho	tempo de resposta, RAM, CPU, bateria e tamanho de instalação
Confiabilidade	taxa de tarefas concluídas e falhas recuperáveis
Segurança	nenhum segredo exposto e permissões auditáveis
Conteúdo	cada episódio produz release/tag funcional e documentação

23. Baseline de versões verificadas em 29/08/2026

Tecnologia	Versão-base	Uso no LZ Agent
Python	3.14.7	núcleo IA, automação, scripts e Blender
.NET runtime	10.0.11	runtime LTS para módulos .NET
.NET SDK	10.0.400	build Windows/.NET
C#	14.0	aplicação e módulos Windows
Windows App SDK	2.4.0	WinUI 3 / integração Windows
Windows SDK	10.0.28000.2705	SDK Windows 11 - linha mais recente publicada em junho/2026
Android Studio	Quail 3 2026.1.3 Patch 1	IDE Android
Android Gradle Plugin	9.3.2	build Android
Kotlin	2.4.10	Android
Flutter	3.47.0	UI Linux cross-distro
Dart	3.13.0	linguagem Flutter
Rust	1.98.0	módulos críticos e integrações
FastAPI	0.141.1	API local/serviços Python
SQLite	3.53.4	banco local padrão
PostgreSQL	18.6	sincronização/servidor opcional
Blender	5.2.1 LTS	modelagem, rig, animação e automação 3D
FFmpeg	9.0.1	áudio/vídeo e conversões
Git	2.55.0	versionamento, checkpoints e rollback
Flatpak	1.18.x estável / versão segura da distro	empacotamento Linux; respeitar backports de segurança
WCAG	2.2 AA	referência de acessibilidade digital
OpenAI - modelo geral	gpt-5.6-sol (alias gpt-5.6)	raciocínio complexo, multimodalidade e orquestração; configurável por provedor
OpenAI - Codex	gpt-5.3-codex	tarefas de programação agentic, revisão e implementação assistida
Gradle	9.5.0	build Android recomendado para AGP 9.3.x
JDK Android	17	baseline exigida pela linha AGP 9.3
Android SDK Build Tools	36.0.0	baseline padrão documentada para AGP 9.3

Regra de atualização: estas versões são um snapshot de 29/08/2026. O repositório fixará versões testadas no lockfile/baseline. Versão mais nova não será instalada automaticamente apenas por ser mais recente; primeiro o agente cria branch/checkpoint, avalia notas de release, executa migrações e testes, e então apresenta a atualização para aprovação.

24. Referências técnicas atuais para iniciar a série

- Microsoft Learn — .NET em Linux para módulos compartilhados e serviços compatíveis quando apropriado.
- Flutter Documentation — suporte e configuração de aplicações desktop Linux.
- Flatpak Documentation — empacotamento cross-distro, runtimes, sandbox e XDG Desktop Portals.
- Microsoft Learn — Windows App SDK / WinUI 3: base moderna para aplicativo Windows.
- Android Developers — Jetpack Compose, arquitetura e acessibilidade.
- W3C — WCAG 2.2 como referência internacional de acessibilidade digital.
- OpenAI — documentação do Codex para assistência de engenharia.
- Blender — Python API para automação do pipeline 3D.
- ONNX Runtime ou runtime equivalente — opção de inferência multiplataforma quando adequada.

Observação de versão: o documento foi atualizado em 29/08/2026 com versões estáveis verificadas em fontes oficiais. Pré-releases como .NET 11 Preview, Kotlin RC, Android 17 Beta e PostgreSQL 19 Beta não serão a base de produção. Antes de cada episódio, o arquivo technology-baseline.json deve ser comparado novamente com as fontes oficiais e atualizado somente após testes.

- Python.org — Python 3.14.7 (05/08/2026).
- Microsoft .NET — .NET 10.0.11 / SDK 10.0.400 (11/08/2026).
- Microsoft Learn — Windows App SDK 2.4.0 (13/08/2026).
- Android Developers — Android Studio Quail 3 2026.1.3 Patch 1 e AGP 9.3.2.
- Kotlin Documentation — Kotlin 2.4.10.
- Flutter/Dart — Flutter 3.47.0 e Dart 3.13.0 (12/08/2026).
- Rust Blog — Rust 1.98.0 (20/08/2026).
- SQLite — SQLite 3.53.4 (24/07/2026).
- PostgreSQL — PostgreSQL 18.6 (13/08/2026).
- Blender — Blender 5.2.1 LTS (25/08/2026).
- FFmpeg — FFmpeg 9.0.1 (12/08/2026).
- Git — Git 2.55.0; Git for Windows build atual de 20/08/2026.
- FastAPI — FastAPI 0.141.1 (29/07/2026).

25. Roadmap de entregas

Marco	Resultado
M0 — conceito	marca, requisitos, riscos, arquitetura e repositório
M1 — MVP	chat + voz básica + uma ferramenta + banco local de ações + Windows/Android/Linux

Marco	Resultado
M2 — acessibilidade	perfis, leitores, voz, switch, legendas e testes
M3 — plugins	produtividade, dev e Blender
M4 — multimodal	visão/documentos + memória persistente + aprendizado controlado + sugestões
M5 — avatar	2D/3D otimizado e estados animados
M6 — beta	instaladores, telemetria opcional, feedback e compatibilidade
M7 — 1.0	documentação, atualização, segurança e publicação

26. Resultado final esperado

Ao final da primeira temporada, o canal terá um repositório funcional, versões executáveis para Windows, Android e Linux dentro de uma matriz de suporte declarada, banco de dados local com histórico auditável, memória por projeto, aprendizado controlado, motor de sugestões, checkpoints e rollback, documentação, testes, identidade visual, avatar, SDK de plugins e processo contínuo de atualização. O LZ Agent deverá reutilizar conhecimento aprovado de projetos anteriores para ajudar em trabalhos futuros sem retirar do usuário o controle sobre dados e mudanças.

Anexo A - Especificação visual oficial do agente

Esta especificação substitui as artes conceituais antigas como referência final para implementação. Ela preserva a identidade tecnológica criada desde o início, mas elimina elementos obsoletos e define medidas, estados e regras que podem ser testados em Windows, Android e Linux.

Elemento	Especificação canônica
Símbolo principal	núcleo circular/visor tecnológico escuro com luz ciano/azul; letras LZ integradas sem prejudicar leitura
Avatar	robô/entidade tecnológica amigável, compacta, sem aparência agressiva; cabeça/visor como foco visual; corpo opcional
Paleta funcional	fundo escuro opcional, ciano/azul como destaque; estados também usam forma, ícone e texto para não depender de cor
Estados	idle, listening, thinking, acting, speaking, approval, success, warning, error, offline e private
Animação	curta, suave, sem flashes; respeita reduzir movimento e possui equivalente estático
Ícone	versão simplificada do visor/núcleo; legível em tamanhos pequenos e em monocromático
Interface	cards limpos, hierarquia forte, navegação previsível, contraste AA e modo simples/avançado

Elemento	Especificação canônica
Voz visual	onda/anel é indicador secundário; sempre existe texto/semântica acessível
3D	LOD automático e fallback 2D/estático para hardware Lite ou redução de movimento
Marca	LZ Agent + assinatura "Seu agente pessoal inteligente" quando houver espaço suficiente
Asset	Entregas obrigatórias
Logo completa	SVG + PNG 1x/2x/4x + monocromática
Ícone Windows	ICO/MSIX assets conforme pacote e escalas exigidas pela plataforma
Ícone Android	Adaptive Icon: foreground/background + versão monocromática quando suportada
Linux	PNG/SVG em tamanhos de desktop e metadados do pacote/Flatpak
Avatar 2D	PNG/WebP/SVG quando aplicável; estados estáticos
Avatar 3D	GLB/glTF otimizado, materiais, rig, animações, LODs e licença/metadados
Áudio	sons curtos opcionais para estados; nunca a única indicação

Regra de fidelidade: toda tela ou asset novo precisa declarar qual componente oficial está implementando e passar por revisão visual, semântica, acessibilidade, desempenho e compatibilidade antes de entrar na versão principal.

Anexo B - Fontes oficiais verificadas em 29/08/2026

- Python 3.14.7 - <https://www.python.org/downloads/release/python-3147/>
- .NET 10.0.11 / SDK 10.0.400 / C# 14 - <https://dotnet.microsoft.com/download/dotnet/10.0>
- Windows App SDK 2.4.0 - <https://learn.microsoft.com/windows/apps/windows-app-sdk/downloads>
- Windows SDK 10.0.28000.2705 - Microsoft Learn, página "What's new: SDK, WinUI, tools"
- Android Studio Quail 3 | 2026.1.3 Patch 1 - <https://developer.android.com/studio/releases>
- Android Gradle Plugin 9.3.x - <https://developer.android.com/build/releases/agp-9-3-0-release-notes>
- Kotlin 2.4.10 - <https://kotlinlang.org/docs/faq.html>
- Flutter 3.47.0 - <https://docs.flutter.dev/release/release-notes/release-notes-3.47.0>
- OpenAI models - <https://platform.openai.com/docs/models> e <https://developers.openai.com/api/docs/models/all>
- WCAG 2.2 - <https://www.w3.org/TR/WCAG22/>

Regra: versões são um snapshot. Antes de cada episódio ou release, o projeto executa a rotina de verificação, registra a data e só muda a versão fixada após testes de compatibilidade, acessibilidade, banco de dados, desempenho e rollback.

Anexo C - Consolidação completa das decisões desde o início

Este anexo registra, em um único lugar, a evolução do projeto e deixa explícito qual decisão final substitui conceitos anteriores. Quando houver conflito entre uma referência antiga e as seções técnicas atuais, prevalece a especificação atual deste Documento Mestre.

Plataformas: O projeto começou com Windows e Android e foi ampliado para Linux. A especificação final exige uma arquitetura comum e adaptações nativas por plataforma, com matriz de suporte e testes reais.

Linux: Incluído suporte planejado a múltiplas famílias de distribuições, desktops e arquiteturas, usando empacotamento e APIs portáteis sempre que possível, com fallback por capacidade.

Docker/containers: Removidos do projeto como dependência arquitetural. O agente não depende de Docker para funcionar.

Banco de dados: A persistência passa a ser parte central: ações, projetos, decisões, preferências, lições, sugestões, versões, checkpoints, auditoria e memória autorizada são armazenados de forma estruturada.

Aprendizado contínuo: O agente aprende com projetos e resultados anteriores por memória controlada e recuperável; ele não modifica silenciosamente o modelo-base. Sugestões futuras devem indicar a origem da experiência usada.

Mudanças automáticas: Fluxo oficial: analisar -> criar checkpoint -> propor/solicitar aprovação conforme autonomia -> alterar -> mostrar diff -> testar -> registrar resultado -> manter ou executar rollback.

Acessibilidade: Permanece requisito de primeira classe para deficiências visuais, auditivas, motoras, de fala, cognitivas, de aprendizagem e combinações. A referência web é WCAG 2.2 AA, complementada pelas APIs nativas.

Hardware: Em vez de prometer execução idêntica em todo hardware existente, o produto usa perfis Lite, Standard e Pro, detecção de capacidade e fallbacks para CPU, GPU/NPU, nuvem ou recursos reduzidos.

IA: Arquitetura multi-provedor e multimodal, separando raciocínio, visão, fala, memória e execução de ferramentas. Modelos podem ser atualizados sem reescrever o produto.

Codex: Usado como parceiro de engenharia: issue, implementação pequena, revisão de diff, testes, acessibilidade, validação e commit. O repositório é a fonte de verdade.

Avatar e identidade: O agente possui logo, ícone, avatar 2D/3D, animações e estados visuais. Deve existir fallback estático/2D e opção de reduzir movimento.

Modelagem 3D: Pipeline com Blender, scripts Python, rig, materiais, animações, LOD e exportação otimizada para os sistemas suportados.

Série de vídeos: A série cobre do manifesto e ambiente de desenvolvimento até IA, memória, acessibilidade, interfaces, plugins, 3D, segurança, testes, beta, publicação e manutenção.

Versões: O documento mantém um baseline verificado por data e diferencia versão mais recente de versão fixada/testada no projeto. Atualizações entram somente após testes.

Anexo D - Arquivo visual completo do conceito

As imagens abaixo fazem parte do histórico de criação do LZ Agent e devem ser preservadas como referência visual. Elas representam direção de arte, avatar, logo, ícone, interfaces e organização da série. Elementos técnicos presentes em artes antigas que contrariem a arquitetura final - por exemplo Docker - são considerados apenas históricos e NÃO fazem parte da implementação atual.



LZ AGENT

SEU AGENTE PESSOAL INTELIGENTE
PARA TODOS, EM QUALQUER LUGAR!

Windows

Android

Uma série de vídeos completa para ensinar como criar do zero um agente pessoal inteligente, acessível, integrado com todas as tecnologias e funcionalidades atuais e futuras, para ajudar em todos os tipos de empregos, serviços e para todas as pessoas, incluindo pessoas com deficiências.

 **CANAL:** @lzworldsgames



1. VISÃO GERAL DO PROJETO

O LZ Agent é um agente pessoal inteligente, multimodal, acessível e personalizável, que utiliza o poder das melhores tecnologias atuais e futuras para ajudar em todos os tipos de tarefas, empregos e serviços. Ele é projetado para funcionar perfeitamente em Windows e Android, em qualquer dispositivo e hardware, com total acessibilidade para todas as pessoas.

PRINCÍPIOS

-  **Universal:** Funciona em todos os dispositivos
-  **Acessível:** Para todas as pessoas, sem exceção
-  **Inteligente:** IA avançada e aprendizado contínuo
-  **Personalizável:** Adaptável a qualquer necessidade
-  **Seguro:** Privacidade e proteção de dados
-  **Integrado:** Conectado com tudo ao seu redor

2. FUNCIONALIDADES PRINCIPAIS

-  **Assistente Inteligente Multimodal**
Voz, Texto, Imagem, Vídeo e Gestos
-  **Automação de Tarefas**
Execução automática de tarefas repetitivas
-  **Suporte a Todos os Empregos e Serviços**
Soluções para todas as áreas e profissões
-  **Acessibilidade Total**
Recursos para todos os tipos de deficiência
-  **Aprendizado Contínuo**
Atualizações automáticas com novas IAs, tecnologias e linguagens
-  **Integração Total**
Windows, Android, Web, IoT e mais
-  **Personalização Avançada**
Interface, comandos, respostas e mais
-  **Segurança e Privacidade**
Criptografia, controle de dados e permissões

3. TECNOLOGIAS E LINGUAGENS

INTELIGÊNCIA ARTIFICIAL

 OpenAI  Claude  Gemini  Llama ...

LINGUAGENS

 Python  Kotlin  C#  C++  Rust  Dart

DESENVOLVIMENTO

 .NET  TensorFlow  PyTorch  Flutter  Electron ...

BANCO DE DADOS

 SQLite  Firebase  PostgreSQL  MongoDB

3D E MULTIMÍDIA

 Blender  Unity  Unreal  Three.js ...

OUTROS

 Docker  Git  WebRTC  MQTT ...

4. ACESSIBILIDADE TOTAL

O LZ Agent é projetado para ser utilizado por todas as pessoas, incluindo aquelas com deficiências visuais, auditivas, motoras, cognitivas e outras.

-  Leitor de Tela Integrado
-  Reconhecimento de Voz Avançado
-  Comandos por Gestos e Movimentos
-  Alto Contraste e Temas Personalizáveis
-  Legendas Automáticas e Tradução
-  Facilitação de Comunicação (AAC)
-  Navegação Simplificada
-  E muito mais...





5. INTEGRAÇÃO E COMPATIBILIDADE

Funciona em todos os hardwares e dispositivos:

-  Windows: Todos os modelos e fabricantes
-  Android: Todos os modelos e fabricantes
-  Integração com IoT, Smart Devices, Robótica, Impressoras 3D e muito mais
-  Sincronização entre dispositivos
-  Funciona Offline e Online



6. INTERFACE DO AGENTE





7. PERSONALIZAÇÃO E 3D



8. SÉRIE DE VÍDEOS

A série será dividida em módulos completos, do zero ao avançado, cobrindo tudo que é necessário para criar, personalizar e manter o LZ Agent sempre atualizado.

MÓDULOS:

- 01 - Introdução ao Projeto
- 02 - Ambiente de Desenvolvimento
- 03 - Criação da Interface
- 04 - Integração com IA
- 05 - Comandos de Voz e Reconhecimento
- 06 - Automação de Tarefas
- 07 - Acessibilidade
- 08 - Integração com Hardware e IoT
- 09 - Sincronização entre Dispositivos
- 10 - Modelagem 3D do Agente
- 11 - Animações e Interações
- 12 - Publicação e Distribuição
- 13 - Atualizações e Manutenção
- 14 - Projetos Práticos e Exemplos

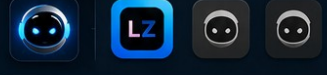


9. LOGO E ÍCONE



LZ AGENT

SEU AGENTE PESSOAL INTELIGENTE



LZ AGENT – INTELIGÊNCIA, ACESSIBILIDADE E INOVAÇÃO PARA TODOS!

CANAL: youtube.com/@lzworldsgames

Figura D.1 - Conceito visual principal do LZ Agent.



Figura D.2 - Painel conceitual histórico de tecnologias, acessibilidade, interfaces, avatar, logo e roteiro. Referências antigas a Docker/containers nesta arte estão revogadas pela arquitetura atual.

Anexo E - Regra de fidelidade do produto final

Para que o aplicativo seja considerado fiel ao Documento Mestre, cada release deverá manter coerência entre: identidade visual, arquivo de configuração canônico, comportamento do modelo,

estados do avatar, banco de dados, memória, permissões, acessibilidade, interfaces Windows/Android/Linux, plugins, segurança, testes e matriz de versões.

- Nenhuma imagem conceitual substitui uma especificação técnica.
- Nenhuma atualização de biblioteca substitui automaticamente a versão validada do projeto.
- Nenhuma memória é permanente sem política de retenção e possibilidade de exclusão.
- Nenhuma automação destrutiva é executada sem a política de autorização correspondente.
- Nenhum recurso visual pode ser a única forma de comunicar estado importante.
- Toda divergência entre plataforma deve preservar a mesma intenção funcional e de acessibilidade.